

# **MAZE SOLVER USING A\* SEARCH ALGORITHM WITH MANHATTAN DISTANCE HEURISTIC**

## **PROJECT REPORT**

Student Name: E SOWMYA  
Roll No.: 07  
Department: BCA  
College: K LH  
Academic Year: 2026–2027

# CERTIFICATE

This is to certify that the project titled "Maze Solver Using A\* Search Algorithm with Manhattan Distance Heuristic" has been completed as part of the academic requirements.

# DECLARATION

I hereby declare that this project report is my work and has been prepared for academic purposes. The implementation, testing, and documentation were carried out as part of the project work.

# ACKNOWLEDGEMENT

I sincerely thank my faculty members and institution for providing guidance and support for completing this project.

# ABSTRACT

This project presents a Maze Solver based on the A\* Search Algorithm. A grid maze is represented using start, goal, open cells, and blocked cells. Manhattan Distance is used as the heuristic because movement is restricted to four directions. The system finds a shortest path when one exists and reports an unreachable case when no path is possible.

## 1. INTRODUCTION

Path finding is an important problem in artificial intelligence, robotics, games, and navigation systems. A\* is an informed search algorithm that uses the cost travelled and an estimate of the remaining distance.

## 2. PROBLEM STATEMENT

Develop a Python program that finds a shortest path from a specified start cell to a goal cell in a grid maze while avoiding blocked cells and handling unreachable goals.

## 3. OBJECTIVES

- Implement A\* Search in Python.
- Use Manhattan Distance as the heuristic.
- Find a shortest path through a grid maze.
- Avoid blocked cells.
- Display the resulting path clearly.
- Handle unreachable goals.

## 4. PROPOSED SYSTEM

The maze is represented as a two-dimensional list. A priority queue stores candidate cells according to their estimated total cost. Four valid neighboring cells are examined until the goal is reached or no candidates remain.

## 5. METHODOLOGY

Create the grid maze, locate start and goal, calculate Manhattan Distance, explore cells using A\*, store parent information, reconstruct the route, display the shortest path, and report an unreachable case if needed.

## 6. A\* ALGORITHM

A\* evaluates each candidate using  $f(n) = g(n) + h(n)$ , where  $g(n)$  is the cost from the start and  $h(n)$  is the estimated cost to the goal.

## 7. MANHATTAN DISTANCE

$h(n) = |x_1 - x_2| + |y_1 - y_2|$ . It is suitable when diagonal movement is not allowed.

## 8. IMPLEMENTATION

The project is implemented in Python using the `heapq` module for the priority queue and `Matplotlib` for visualization. The maze uses S for start, G for goal, # for walls, . for open cells, and \* for the path.

## 9. RESULTS

The program identifies start and goal positions, computes a shortest path when available, prints path coordinates and path length, and visualizes the route. A separate test demonstrates the no-path condition.

## 10. ADVANTAGES

- Simple and easy to understand.
- Finds a shortest path for the chosen movement model.
- Uses an informed heuristic.
- Handles blocked cells.
- Can be visualized easily.

## 11. LIMITATIONS

- Fixed maze in the current version.
- Four-direction movement only.
- Basic visualization.

## 12. FUTURE SCOPE

Interactive maze creation, random maze generation, animated search, diagonal movement, weighted terrain, GUI, and comparison with BFS and Dijkstra's algorithm.

## 13. CONCLUSION

The project demonstrates how A\* Search can solve a grid-based maze using Manhattan Distance as a heuristic. It finds a shortest path while avoiding walls and reports when a goal is unreachable.

## 14. REFERENCES

1. Python Documentation – `heapq`: Heap Queue Algorithm.
2. Matplotlib Documentation – Python plotting library.
3. GeeksforGeeks – A\* Search Algorithm.
4. GeeksforGeeks – Manhattan Distance.
5. Streamlit Documentation – App development and deployment.